

HTML & CSS

1. What are FCP and LCP?
2. How can we improve the FCP performance?
3. If we need to serve files or assets to users in Bangladesh from a Bangladesh server and users in the USA from a USA server, how can we achieve this?
4. How can we create a pulse animation effect for a button?
5. Suppose you have two `<div />` elements where the top `<div>` has `margin-bottom: 50px`, and the bottom `<div>` has `margin-top: 50px`. What will be the total space between them? What if the top `<div>` has `margin-bottom: 50px` and the bottom `<div>` has `margin-top: 20px`?
6. If two `<div>` elements are placed side by side, and the left `<div>` has `margin-right: 50px`, while the right `<div>` has `margin-left: 50px`, what will be the total space between them?
7. What are inline, inline-block, and block elements? How do they differ?
8. How do we load a JavaScript script?
9. What is the difference between loading a script in the `<head>` tag versus at the bottom of the `<body>` tag?
10. Given an SVG with `viewBox="0 0 16 16"`, how can we modify its height?
11. What are Stroke and Fill in SVG, and how do they work?
12. What are `vh` and `dvh` units? What problem does `dvh` solve?
13. Why do we use CSS Grid instead of Flexbox? Since Flexbox can solve many layout problems, what advantages does Grid offer?
14. If you have a large image, what are some ways to optimize its loading speed?
15. How can we ensure the last message in a chat box is initially visible using CSS? (Hint: `flex-direction: column-reverse`)
16. What are the `:has()` and `:is()` pseudo-classes in CSS, and how do they work?
17. Have you ever customized Tailwind CSS using its configuration file?
18. What are combinator selectors (`+`, `>`, `~`) in CSS and how do they work?
19. Have you used Bootstrap in your projects?

JS

1. If you have five buttons and more buttons are dynamically added from an API, how can you attach an event listener to them efficiently?
2. When using the Event Delegation technique, what will be the difference between `event.currentTarget` and `event.target`?
3. Suppose you have a card component with a button inside it, and both elements have event listeners. How can you prevent the event from bubbling up when clicking the button?

4. What is the concept of Closure in JavaScript?
5. Where should we store authentication tokens securely?
6. If we store tokens in cookies, they can still be accessed using `document.cookie`. Is there a way to prevent this?
7. If `var` can declare variables, why should we use `let` and `const` instead?
8. Tell the output of the following code and explain the reason:

```
const a = {  
  name: "A",  
  address: {  
    name: "AAA",  
  },  
};  
  
const b = { ...a };  
b.name = "B";  
b.address.name = "BBB";  
  
console.log(a.name);  
console.log(b.name);  
console.log(a.address.name);  
console.log(b.address.name);
```

React

1. What is a Higher-Order Component (HOC), and what purpose does it serve?
2. How can we pass data from a child component to a parent component in React?
3. What is the Render Props pattern in React?
4. What is `useLayoutEffect` in React? Provide an example of its usage.
5. Why do we use the `key` prop when rendering a list in React?
6. What is the difference between a controlled and uncontrolled component?

Next.js and NextAuth

1. If you need to store user permissions in the session, how can you do it?
2. How do you refresh an authentication token and update the session accordingly?
3. In Next.js, every request goes through something. What is this called?
4. How can we refresh the authentication token inside middleware in Next.js?

Typescript

1. What is the difference between `type` and `interface` in TypeScript?
2. Suppose you have two types, `type User {...}` and `type Driver {...}`. How can you create a new type `type DriverUser {...}` that combines all the properties of `User` and `Driver`?
3. If we use `interface` instead of `type`, how can we achieve the same result?

React Native

1. How do we store tokens?

JS

1. Event loop (callback queue, macrotask queue, microtask queue) [JavaScript Visualized - Event Loop, Web APIs, \(Micro\)task Queue](#)
2. Event delegation [Mastering JavaScript Event Delegation: Simplifying DOM Interaction for Web Developers!](#)
3. Event propagation, bubbling & capturing [Mastering JavaScript Event Propagation - Bubbling vs. Capturing Explained](#)
4. Promise [JavaScript Visualized - Promise Execution](#)
5. Async / await
6. OOP in JS
7. IntersectionObserver and MutationObserver
8. JS Engine
9. Execution context [What is JavaScript Execution Context ? | Call Stack & Heap Memory](#)
10. Hoisting [Hoisting এটা আবার কি 🤔? | Hoisting in JavaScript](#)
11. Closure
12. Curry function
13. Prototype
14. Prototypal inheritance
15. Type coercion
16. Symbol
17. "new" keyword for creating instances
18. "this" keyword
19. More questions: [50 Vital Advanced JavaScript Interview Questions & Answers!](#)
- 20.